

Unit 1: Number System and Logic Gates

Chapter 1: Introduction to Number Systems

1.1 What is a Number System?

A **Number System** is a structured way to represent numbers using a specific set of symbols and rules. In simple words, it is a **mathematical language** that helps us count, measure, and perform calculations.

Think of it like this: just as we use the English alphabet to write words, we use a number system to write values.

1.2 Why is the Number System Important in Computers?

This is the most important thing to understand:

Every piece of data in a computer—whether it is text, images, sound, video, or code—is ultimately stored as binary (a string of only 0s and 1s).

Computers are electronic machines. Inside a computer, there are millions of tiny switches that can only be in **two states**:

- **ON** (represented as 1)
- **OFF** (represented as 0)

Because of this physical limitation, computers only understand the **Binary Number System** (which uses only 0 and 1).

1.3 The Big Difference: Human vs. Computer

There is a gap between how humans and computers understand numbers:

Humans	Computers
Understand the Decimal System (0 to 9) easily.	Understand only the Binary System (0 and 1).
Find long binary strings (like 1011011101) hard to read.	Process binary strings very fast.

To bridge this gap, computer scientists created two more number systems:

- **Octal** (Base 8)
- **Hexadecimal** (Base 16)

These systems allow us to write long binary numbers in a much **shorter, readable, and error-free** way.



Figure 1: How Data is Stored in a Computer

The diagram below shows how everything we see on a screen is finally converted into binary inside the computer's memory.

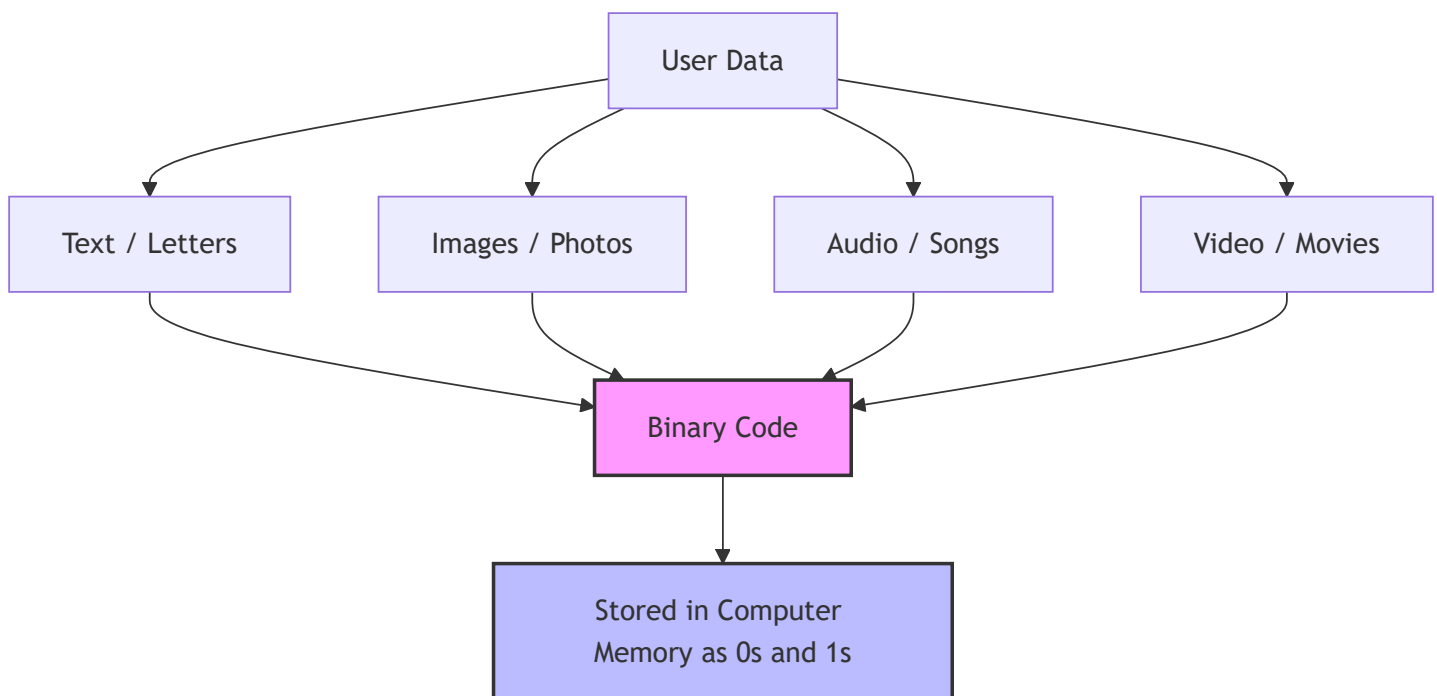


Figure 1: All types of data (text, images, sound, and video) are ultimately converted into binary (0s and 1s) to be stored and processed by the computer.

1.4 The Concept of Base (or Radix)

Every number system has a **base** (also called **radix**).

The **base** tells us two things:

1. How many unique digits are used in that system.
2. The place value (weight) of each position in a number.

For example:

- If the base is **10**, we use 10 digits (0 to 9).
- If the base is **2**, we use only 2 digits (0 and 1).

1.5 Types of Number Systems

There are four main types of number systems used in computer science:

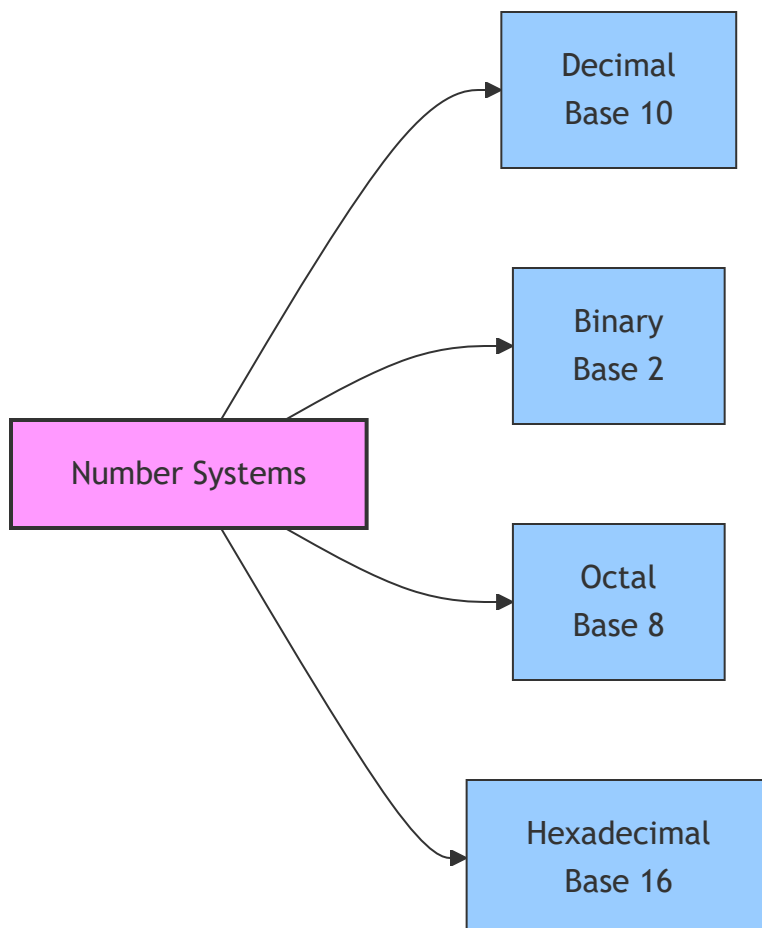


Figure 2: Classification of Number Systems used in computers.

Let us study each of these systems in detail.

1. Decimal System (Base 10)

The Decimal System is the number system we use in everyday life.

- **Base:** 10
- **Digits used:** 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- **Place Value:** Each position has a weight of a power of 10 (1, 10, 100, 1000, ...).

Example:

The number **4307** means:
 $(4 \times 1000) + (3 \times 100) + (0 \times 10) + (7 \times 1) = 4000 + 300 + 0 + 7 = 4307$.

2. Two-State Devices (The Heart of Computers)

Computers are electronic machines. They are made of millions of tiny electronic switches called **Transistors**.

These switches are **Two-state devices** because they can exist in only two physical states:

State	Electrical Condition	Representation
State 1	Switch is ON (High Voltage, e.g., 5 Volts)	1
State 0	Switch is OFF (Low Voltage, e.g., 0 Volts)	0

Because of this hardware limitation, computers can only understand and process **two symbols**: 0 and 1. This is why the **Binary System** is the native language of computers.

 Visual Idea: The Light Switch Analogy

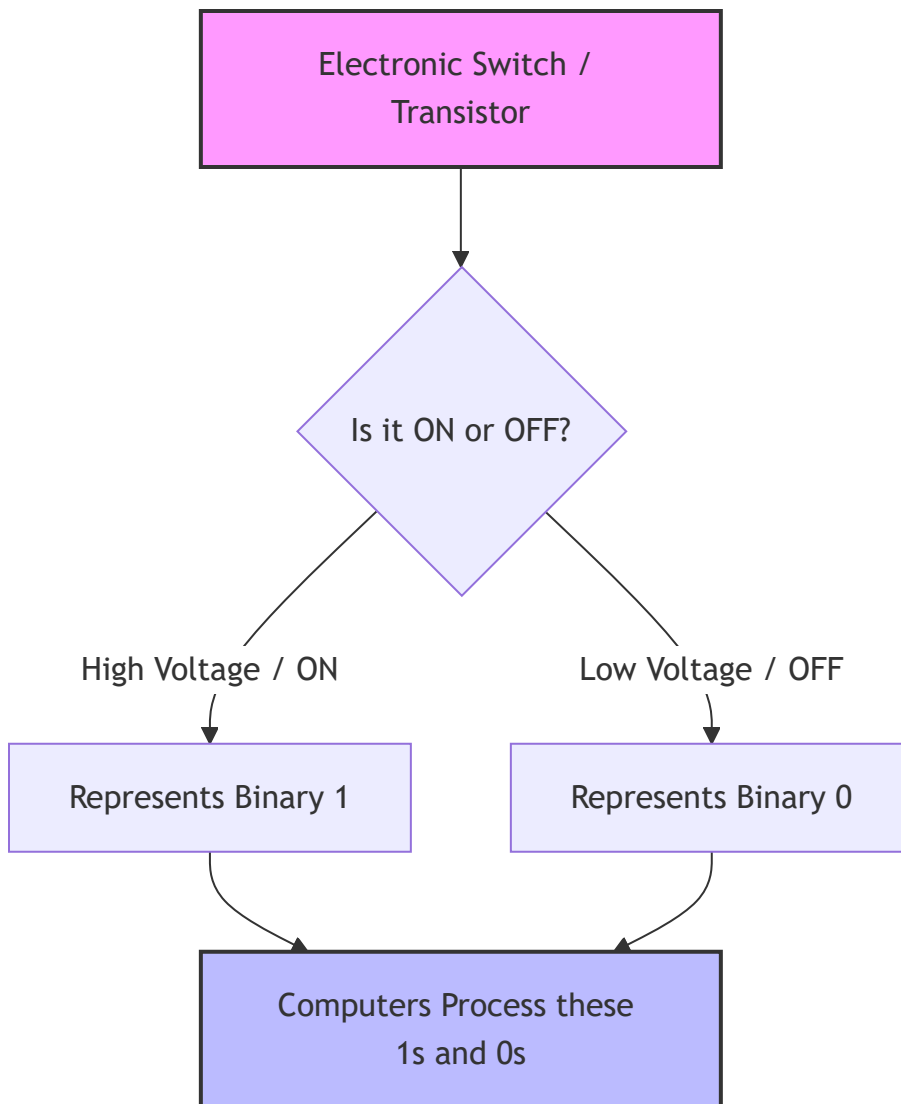
Think of a regular room light switch:

- **Switch Up (ON)** → Light glows → represents **1**

- **Switch Down (OFF)** → Light is off → represents **0**

Now imagine millions of these switches inside your computer. By turning them ON and OFF in different patterns, the computer represents all types of data.

Two-State Device Concept



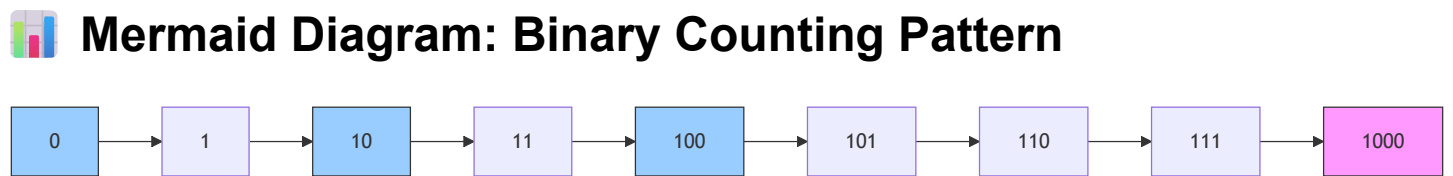
3. Counting in Binary System

Since we only have two digits (0 and 1), counting in binary is very simple but works exactly like decimal counting.

The Rule: When you reach the highest digit (which is 1), you reset it to 0 and carry 1 to the next position on the left.

Let's count from 0 to 15 in binary and see the pattern:

Decimal Number	Binary Number	Why? (Place Values)
0	0	Starting point.
1	1	One.
2	10	We reached '1', so we carry over. ($1 \times 2^1 + 0 \times 2^0 = 2$)
3	11	($1 \times 2^1 + 1 \times 2^0 = 3$)
4	100	Reached '11', carry over to next place. (1×4)
5	101	($1 \times 4 + 0 \times 2 + 1 \times 1$)
6	110	($1 \times 4 + 1 \times 2 + 0 \times 1$)
7	111	($1 \times 4 + 1 \times 2 + 1 \times 1$)
8	1000	Reached '111', carry over to 8s place.
9	1001	
10	1010	
11	1011	
12	1100	
13	1101	
14	1110	
15	1111	



4. Binary Addition and Subtraction

4.1 Binary Addition

Binary addition is very easy. You just need to remember 4 simple rules:

Rule	Result	Carry
0 + 0	0	0
0 + 1	1	0
1 + 0	1	0
1 + 1	0	1 (carry to next column)

Example: Add $(101)_2$ and $(111)_2$ together.

Step	Column (2^2)	Column (2^1)	Column (2^0)	Explanation
Number 1	1	0	1	
Number 2	1	1	1	
Addition				
Step 1 (Rightmost)			1 + 1 = 0, Carry 1	(Rule 4)
Step 2 (Middle)		0 + 1 + 1 = 0, Carry 1		(Rule 4 with carry)
Step 3 (Leftmost)	1 + 1 + 1 = 1, Carry 1			(Three ones make 11 in binary)
Final Result	1	1	0	

Answer: $(101)_2 + (111)_2 = (1100)_2$

4.2 Binary Subtraction

Binary subtraction also has 4 simple rules:

Rule	Result	Borrow
0 - 0	0	0
1 - 0	1	0
1 - 1	0	0
0 - 1	1	1 (borrow from next column)

Example: Subtract $(0110)_2$ from $(1010)_2$.

Step	Column (2^3)	Column (2^2)	Column (2^1)	Column (2^0)	Explanation
Number 1	1	0	1	0	
Number 2	0	1	1	0	
Subtraction					
Step 1 (Rightmost)				0 - 0 = 0	
Step 2 (Middle-2)			1 - 1 = 0		
Step 3 (Middle-1)		0 - 1 = 1 (Borrow 1)			We borrow from the left '1'.
Step 4 (Leftmost)	1 - 0 - 1 = 0				The '1' became 0 after borrowing.
Final Result	0	1	0	0	

Answer: $(1010)_2 - (0110)_2 = (0100)_2$ which is 4 in decimal.

5. Converting Decimal Number to Binary Numbers

To convert a decimal number to binary, we use the **Repeated Division by 2** method.

Steps:

1. Divide the decimal number by 2.
2. Write down the **remainder** (it will always be 0 or 1).
3. Divide the quotient again by 2.
4. Keep repeating until the quotient becomes 0.
5. Read the remainders **from bottom to top** (reverse order).

Example: Convert $(13)_{10}$ to Binary

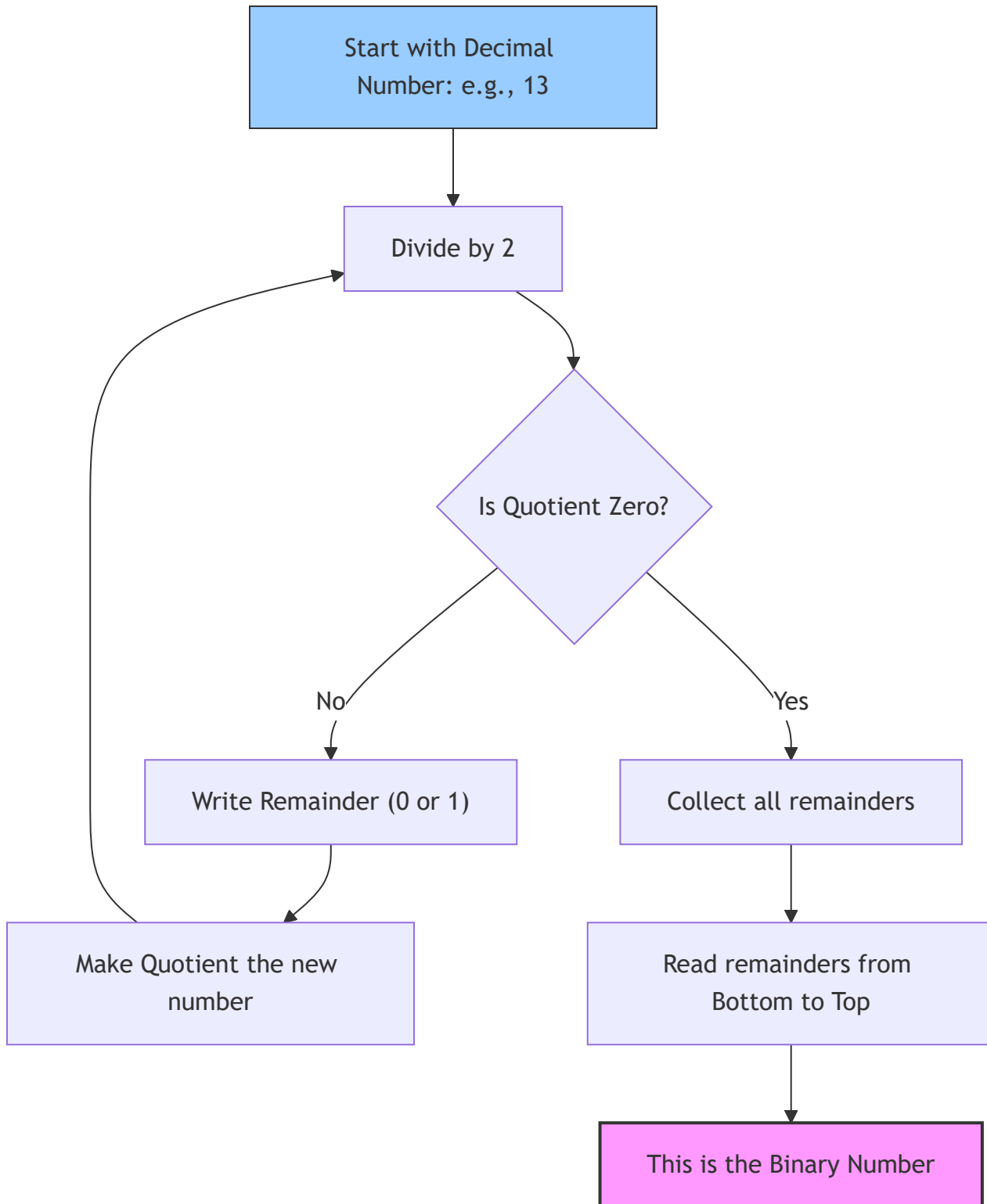
Step	Division	Quotient	Remainder	Bit
1	$13 \div 2$	6	1	Least Significant Bit (LSB)
2	$6 \div 2$	3	0	
3	$3 \div 2$	1	1	
4	$1 \div 2$	0	1	Most Significant Bit (MSB)

Now, read the remainders from **bottom to top**: **1101**.

Answer: $(13)_{10} = (1101)_2$



Decimal to Binary Conversion Flowchart



6. Use of Complements to Represent Negative Numbers

Computers cannot understand a minus (-) sign. They only understand 0s and 1s. So, how do we represent negative numbers like -5?

The answer is: **Complements**.

There are two types of complements used in binary:

1. **1's Complement**
2. **2's Complement**

6.1 1's Complement

To get the 1's complement of a binary number, simply **flip (invert) all the bits**.

- Change all 0s to 1s.
- Change all 1s to 0s.

Example: Find the 1's complement of $(1010)_2$.

Original Bits	1	0	1	0
1's Complement	0	1	0	1

So, 1's complement of 1010 is **0101**.

6.2 2's Complement (The Standard Method)

Computers use the **2's Complement** method to store negative numbers. It is very simple:

2's Complement = 1's Complement + 1

Steps to find 2's complement:

1. Write the number in binary.
2. Find the 1's complement (flip all bits).
3. Add 1 to the result.

Example: Represent (-5) in Binary using 2's Complement

Assume we are using 4 bits.

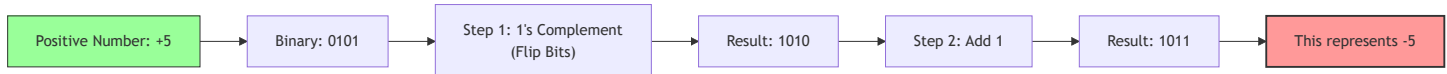
Step	Action	Result
1	Write +5 in binary	0101
2	Find 1's complement (flip bits)	1010
3	Add 1 to the 1's complement	$1010 + 0001 = \mathbf{1011}$

Therefore: In a 4-bit computer, $(1011)_2$ represents **-5**.

Important: The leftmost bit (Most Significant Bit) tells us the sign.

- If it is **0**, the number is **positive**.
- If it is **1**, the number is **negative**.

How 2's Complement Works



7. Octal and Hexadecimal Number Systems

Both Octal (Base 8) and Hexadecimal (Base 16) are used to make **long binary strings shorter and easier for humans** to read.

7.1 Why do we group binary?

System	Grouping Rule	Reason
Octal	Group binary digits into groups of 3 (bits)	$2^3 = 8$, so one octal digit = 3 binary digits.
Hexadecimal	Group binary digits into groups of 4 (bits)	$2^4 = 16$, so one hex digit = 4 binary digits.

7.2 Converting Binary to Octal (Group of 3)

Example: Convert $(101101)_2$ to Octal.

Step	Action	Result
1	Write the binary number	101101
2	Group from right to left into 3s	101 101
3	Convert each group to decimal (0-7)	$101 = 5$, $101 = 5$

Answer: $(101101)_2 = (55)_8$

7.3 Converting Binary to Hexadecimal (Group of 4)

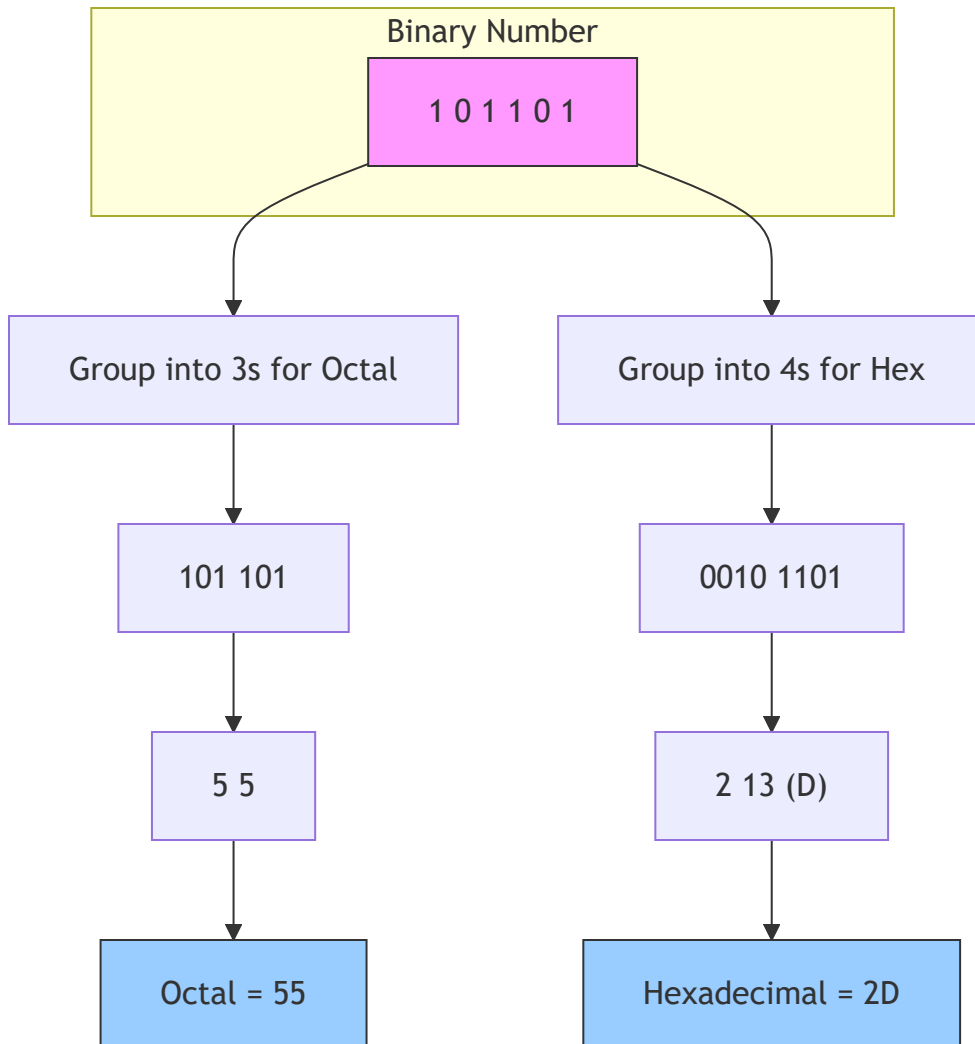
Example: Convert $(101101)_2$ to Hexadecimal.

Step	Action	Result
1	Write the binary number	101101
2	Add leading zeros to make a group of 4	0010 1101 (Added two zeros on left)
3	Convert each group to decimal (0-15)	$0010 = 2$, $1101 = 13$ (which is D in Hex)

Answer: $(101101)_2 = (2D)_{16}$



Binary to Octal/Hex Grouping



7.4 Converting Octal/Hex to Decimal (Quick Recap)

You can always convert Octal or Hexadecimal to decimal using the place-value method.

Octal Example: $(325)_8$

$$= (3 \times 8^2) + (2 \times 8^1) + (5 \times 8^0)$$

$$= (3 \times 64) + (2 \times 8) + (5 \times 1)$$

$$= 192 + 16 + 5 = \mathbf{(213)_{10}}$$

Hex Example: $(2F)_{16}$

$$= (2 \times 16^1) + (15 \times 16^0) \text{ (Since } F = 15)$$

$$= 32 + 15 = \mathbf{(47)_{10}}$$



Final Summary Table for All Topics

Topic	Key Point
Decimal System	Base 10, used by humans.
Two-state Devices	Transistors have ON(1) and OFF(0) states.
Counting in Binary	Follows carry-over rule just like decimal, but max digit is 1.
Binary Addition	$1+1 = 0$ with a carry of 1.
Binary Subtraction	$0-1$ requires borrowing from the next column.
Decimal to Binary	Repeated division by 2, read remainders upward.
1's Complement	Flip all bits ($0 \rightarrow 1$, $1 \rightarrow 0$).
2's Complement	1's complement + 1. Used for negative numbers.
Octal System	Base 8, groups binary in 3s.
Hexadecimal System	Base 16, groups binary in 4s.

Chapter 2: Basic Logic Gates

1. What are Logic Gates?

A **Logic Gate** is a small electronic circuit that takes one or more inputs (0 or 1) and produces a single output (0 or 1).

Think of a logic gate like a **security guard** at the entrance of a building. The guard checks the rules (inputs) and then decides whether to let you in (output 1) or keep you out (output 0).

Computers use millions of these gates to perform all calculations. Logic gates work on **Boolean Algebra**, which uses only two values: **True (1)** and **False (0)**.

2. Logical Multiplication (AND Operation)

In Boolean algebra, multiplication does not mean regular math multiplication.

Logical Multiplication means: "**All inputs must be TRUE (1) for the output to be TRUE (1).**"

The symbol for logical multiplication is a **dot (·)** or just writing letters together (like **AB**).

Rule: The output is 1 **ONLY IF** all inputs are 1. If even one input is 0, the output is 0.

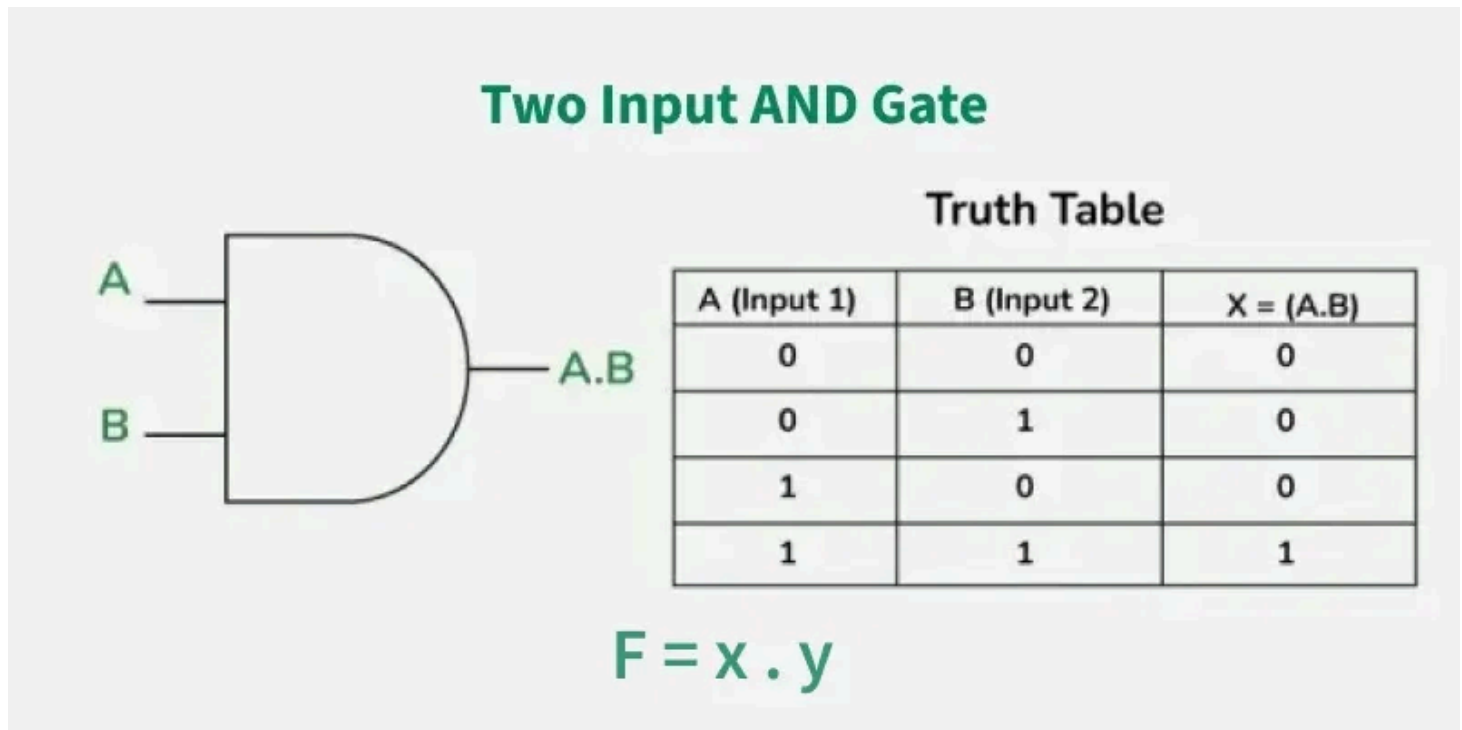
3. AND Gate

The AND Gate performs Logical Multiplication.

Key Features:

- It has **2 or more** inputs.
- It has **1** output.
- It is represented by the equation: **$Y = A \cdot B$** (reads as "Y equals A AND B").

Figure 1: Diagram of AND Gate



Example of AND Gate in Real Life

Imagine a **room with two doors**. A burglar alarm rings only if:

- Door 1 is OPEN (Input A = 1) **AND**
- Door 2 is OPEN (Input B = 1)

If only one door is open, the alarm does NOT ring. The alarm rings (Output = 1) only when **both** doors are open at the same time.

4. OR Gate

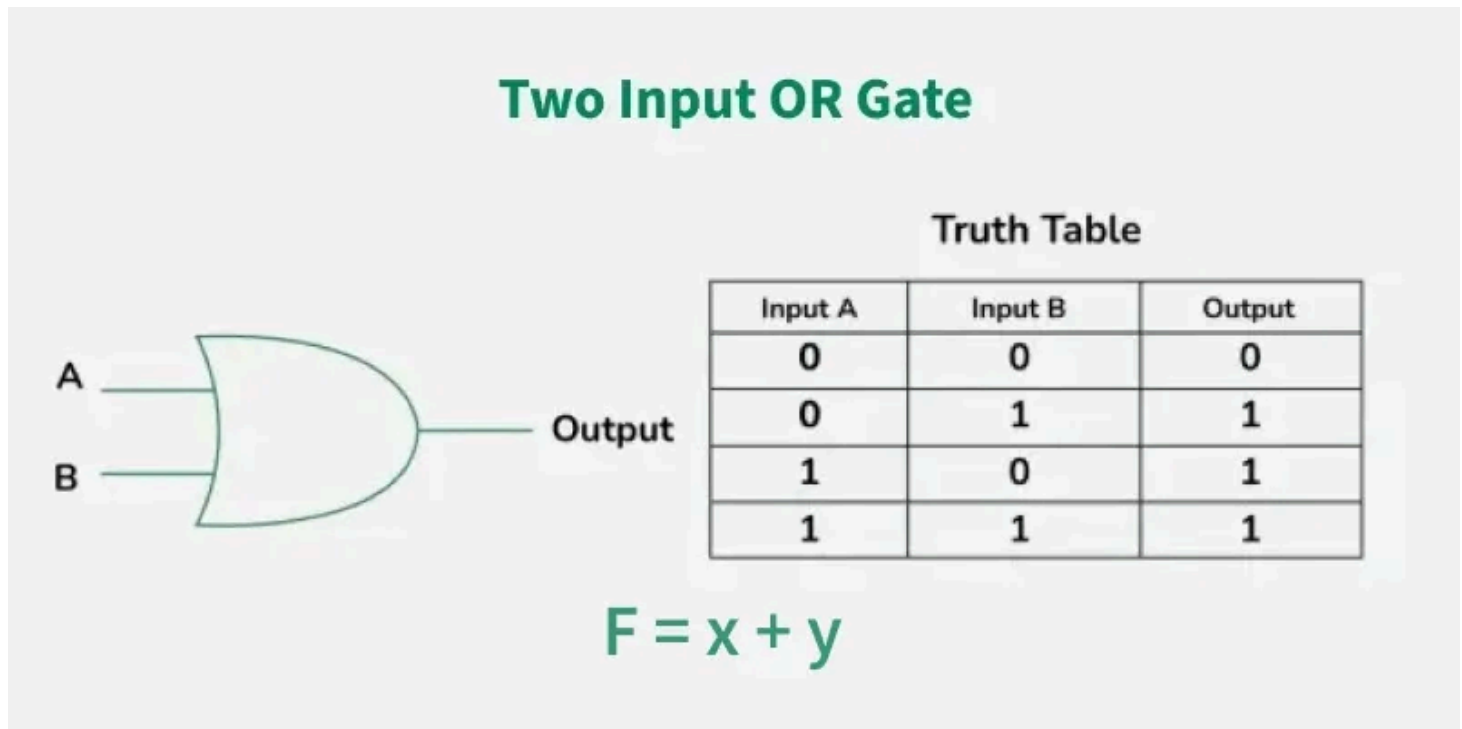
The OR Gate performs Logical Addition.

Rule: The output is 1 **IF ANY ONE** of the inputs is 1.

Key Features:

- It has **2 or more** inputs.
- It has **1** output.
- It is represented by the equation: $Y = A + B$ (reads as "Y equals A OR B").

Figure 2: Diagram of OR Gate



Example of OR Gate in Real Life

Imagine you are applying for a **discount coupon** at a store. You get the discount if:

- You have a Student ID (Input A = 1) **OR**
- You have a Senior Citizen Card (Input B = 1)

You don't need both. If you have **at least one**, the cashier gives you the discount (Output = 1).

5. Complementation and Inverts (NOT Gate)

Complementation means **inverting** or **reversing** the value. The NOT Gate is also called an **Inverter** because it changes 0 to 1, and 1 to 0.

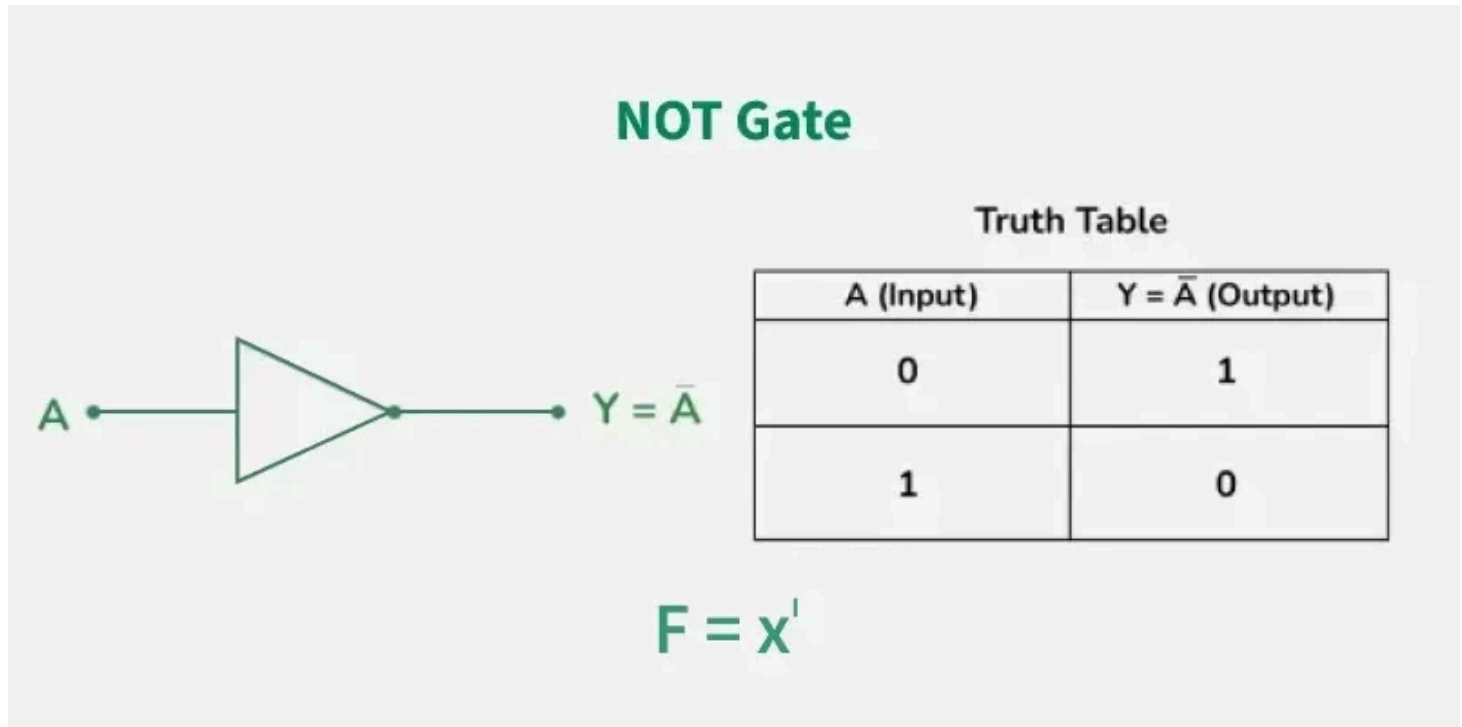
Rule: The output is always the **opposite** of the input.

Key Features:

- It has only **1** input.
- It has **1** output.

- It is represented by the equation: $Y = A'$ or $Y = \bar{A}$ (reads as "Y equals NOT A").

Figure 3: Diagram of NOT Gate



Example of NOT Gate in Real Life

Imagine a **light bulb** connected to a switch. But this is a special switch!

- When you press the switch **UP (1)**, the light turns **OFF (0)**.
- When you press the switch **DOWN (0)**, the light turns **ON (1)**.

It always does the opposite of what you tell it!

6. Evaluation of Logical Expressions

An expression combines multiple gates. To evaluate it, you must know the values of all inputs and then solve step-by-step.

The Golden Rule: Always solve **multiplication (AND)** before **addition (OR)** unless there are parentheses. (Just like in normal math: multiplication comes before addition).

Example: Evaluate $Y = A \cdot B + C$

Let us take: $A = 1$, $B = 0$, $C = 1$

Step	Action	Calculation	Result
1	Solve the AND part first ($A \cdot B$)	$1 \cdot 0$	0
2	Now solve the OR part ($+ C$)	$0 + 1$	1

Final Answer: $Y = 1$

Example: Evaluate $Y = A \cdot B + C \cdot D$

Let us take: $A = 1$, $B = 1$, $C = 0$, $D = 1$

Step	Action	Calculation	Result
1	Solve first AND ($A \cdot B$)	$1 \cdot 1$	1
2	Solve second AND ($C \cdot D$)	$0 \cdot 1$	0
3	Solve the OR between the two results	$1 + 0$	1

Final Answer: $Y = 1$

7. Evaluation of Expressions Containing Parenthesis

Parentheses change the order of evaluation. **Rule:** Always solve the expression **inside the parentheses () first**, just like in regular mathematics.

Example: Evaluate $Y = (A + B) \cdot C$

Let us take: $A = 0$, $B = 1$, $C = 0$

Step	Action	Calculation	Result
1	Solve inside the parenthesis (A + B) first	$0 + 1$	1
2	Now multiply the result by C	$1 \cdot 0$	0

Final Answer: $Y = 0$



Example: Evaluate $Y = A \cdot (B + C) \cdot D$

Let us take: $A = 1, B = 0, C = 1, D = 0$

Step	Action	Calculation	Result
1	Solve inside the parenthesis (B + C) first	$0 + 1$	1
2	Now multiply left-to-right: $A \cdot (\text{Result})$	$1 \cdot 1$	1
3	Multiply by D	$1 \cdot 0$	0

Final Answer: $Y = 0$

8. NAND Gate

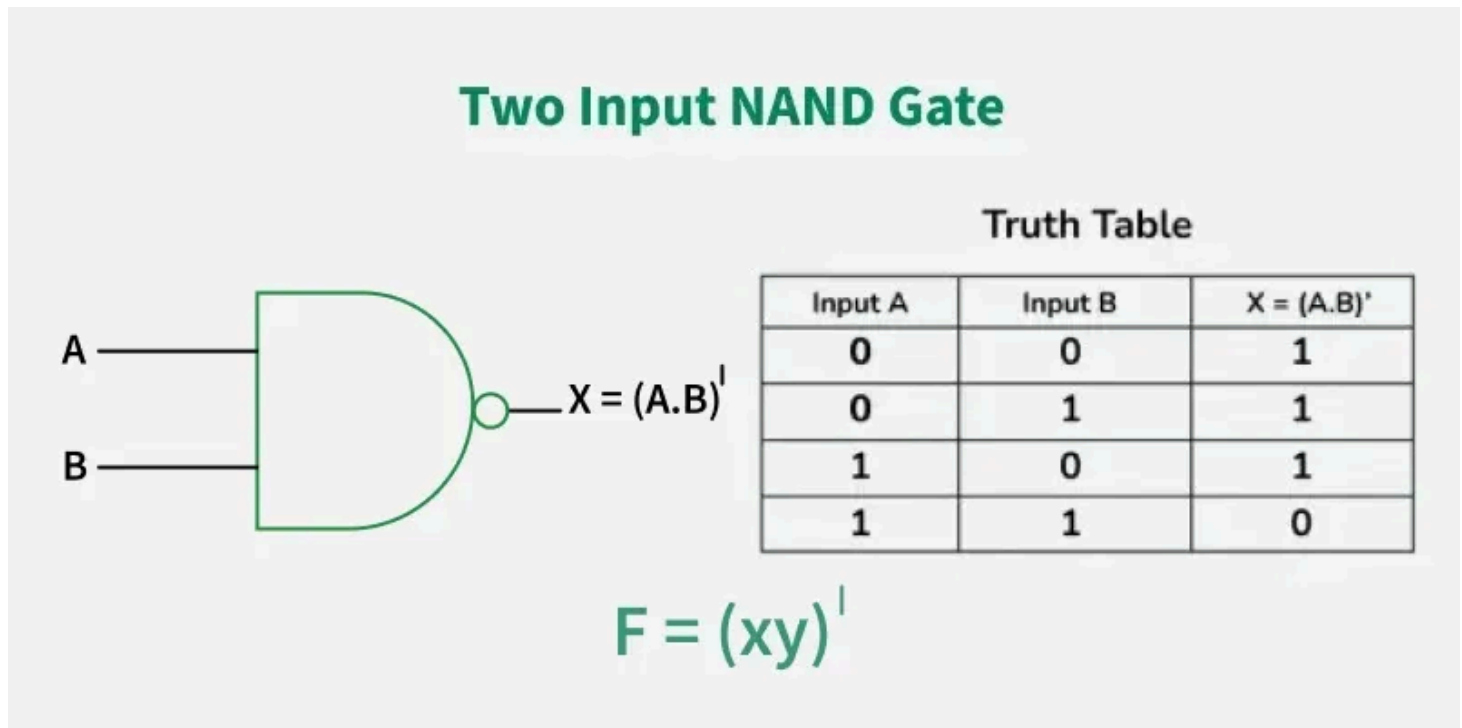
NAND stands for "**NOT AND**". It is made by connecting an AND Gate followed by a NOT Gate (Inverter).

Rule: The output is the **opposite** of the AND Gate.

- It gives **0** only when **ALL** inputs are 1.
- It gives **1** in all other cases.

Equation: $Y = \overline{A \cdot B}$ (Reads as "Y equals NOT of A AND B")

Figure 4: Diagram of NAND Gate



Real-Life Example of NAND Gate

Think of a **house alarm** that is **silent** (0) only when both the Front Door and Back Door are closed (both 1). If *any* door is open, the alarm blares (1). It is the "safety check" gate.

9. NOR Gate

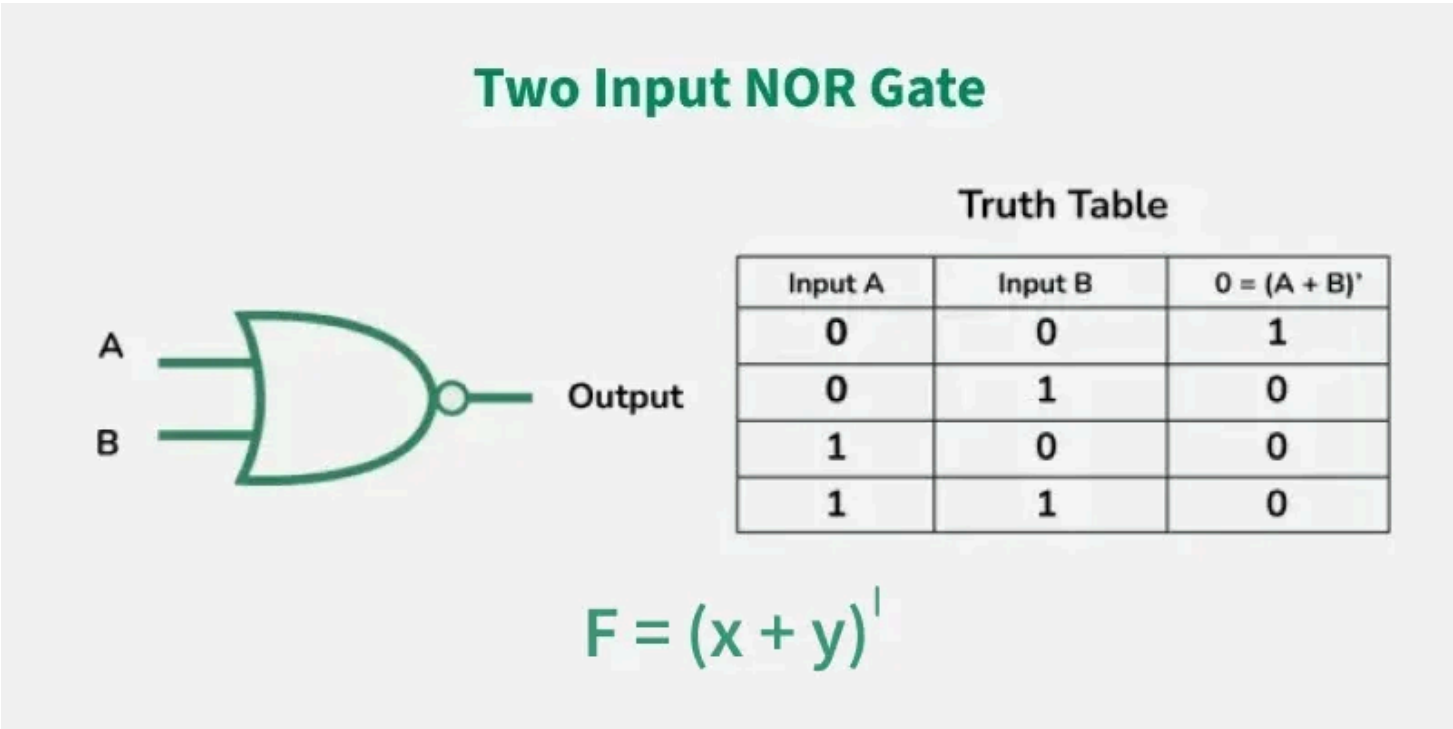
NOR stands for "**NOT OR**". It is made by connecting an OR Gate followed by a NOT Gate (Inverter).

Rule: The output is the **opposite** of the OR Gate.

- It gives **1** only when **ALL** inputs are **0**.
- It gives **0** in all other cases.

Equation: $Y = \overline{A + B}$ (Reads as "Y equals NOT of A OR B")

Figure 5: Diagram of NOR Gate



 Real-Life Example of NOR Gate

Think of a **VIP room** that is **strictly private**. A guard lets a person enter (Output = 1) ONLY if **NO ONE** is inside. If even one person is already inside (Input A or B = 1), the guard blocks the door (Output = 0). It is the "completely empty" check gate.

 Final Summary Table for All Logic Gates

Gate Name	Symbol	Equation	Key Rule	Output is 1 when...
AND	· (dot)	$Y = A \cdot B$	Logical Multiplication	All inputs are 1.
OR	+ (plus)	$Y = A + B$	Logical Addition	Any input is 1.
NOT	Bar ($\bar{}$)	$Y = \bar{A}$	Complementation	Input is 0.

Gate Name	Symbol	Equation	Key Rule	Output is 1 when...
NAND	AND + NOT	$Y = \overline{A \cdot B}$	Opposite of AND	Any input is 0.
NOR	OR + NOT	$Y = \overline{A + B}$	Opposite of OR	All inputs are 0.

Prepared by:

Radhika Zakhaniya (B.C.A)